`CreateModel` class is identified to the outside world by its presence in the class's constructor.

7.2 Inversion of control containers

Of course, in C#, you cannot instantiate an interface, so how can this code make any sense? The actual type passed in to the constructor argument can be any type that implements the specified interface. At run time, an implementation of the interface is provided to the constructor. A built-in dependency injection container provides the implementation. This is more commonly referred to in Microsoft documentation as a *service container*, although more widely, you might also see this type of component referred to as an IoC container or a DI container.

So that leaves just one question: how does the container know which implementation to provide? And the answer is you tell it by registering your services with the container.

7.2.1 Service registration

Service registration takes place in the `Program` class by adding registrations to the `WebApplicationBuilder`'s `Services` property. You may recall seeing this in chapter 2 when I discussed using the `IMiddleware` interface for building middlewares, although I didn't go into detail at the time. The standard web application template already includes a registration for Razor Pages through the `AddRazorPages` method, which is responsible for registering all the services Razor Pages depends on, including those that are responsible for generating and matching routes, handler method selection and page execution, as well as the Razor view engine itself.